

Task Space Trajectory Generation and Tracking Simulation of a 2DoF Robot

KON513E - Final Project
ITU Control and Automation Engineering M.Sc.

June 20, 2026

Abstract

This report details the development of a continuous-time forward dynamics simulation environment in MATLAB for a 2-Degree-of-Freedom (2-DoF) planar robotic manipulator. The dynamic model is derived using the Lagrange-Euler formulation and structured into a state-space representation.

1 Introduction

The primary objective of this project is to develop, implement, and analyze a comprehensive motion control and trajectory generation framework for a robotic manipulator. To rigorously evaluate the control architecture without obfuscating the fundamental system dynamics, a 2-Degree-of-Freedom (2-DoF) planar robot platform is intentionally selected. Restricting the configuration space to $n = 2$ minimizes the algebraic complexity of the manipulator's spatial and inertial matrices. This deliberate simplification facilitates a highly focused investigation into the synthesis and performance of nonlinear control laws and optimal path planning techniques, specifically operational-space feedback linearization and optimized cubic spline trajectory generation.

The theoretical and computational foundation of this framework relies on rigorous kinematic and dynamic modeling. The spatial mapping between the joint space, $q \in \mathbb{R}^2$, and the operational task space, $x \in \mathbb{R}^2$, is established via Forward Kinematics, $x = f(q)$, while the differential motion is governed by Velocity Kinematics, $\dot{x} = J(q)\dot{q}$. To resolve end-effector reference tracking, the Inverse Jacobian, $J^{-1}(q)$, is utilized to map task-space velocities back to the joint space. Furthermore, the physical behavior of the manipulator is characterized through Forward Dynamics and Inverse Dynamics formulations, yielding the standard rigid-body equation of motion:

$$M(q)\ddot{q} + C(q, \dot{q})\dot{q} + g(q) = \tau \quad (1)$$

where $M(q) \in \mathbb{R}^{2 \times 2}$ is the symmetric, positive-definite inertia matrix, $C(q, \dot{q}) \in \mathbb{R}^{2 \times 2}$ represents Coriolis and centrifugal forces, $g(q) \in \mathbb{R}^2$ is the gravity vector, and $\tau \in \mathbb{R}^2$ is the vector of applied joint torques.

Leveraging this dynamic model, the project implements a Computed Torque Controller to exactly linearize and decouple the nonlinear plant dynamics. This methodology is

subsequently extended to a Task Space Controller, enabling the direct regulation of Cartesian end-effector coordinates. To ensure smooth, continuous, and dynamically feasible reference paths, Task Space Trajectory Generation is executed utilizing optimized cubic splines.

Finally, the complete theoretical framework—encompassing kinematics, dynamics, trajectory generation, and control—is validated through the development of a custom simulation environment on MATLAB. This numerical testing environment provides opportunities for controller design and analysis, while also serving as a first step in the development of high-level trajectory planner algorithms. Furthermore, in further studies, a 2DoF robot model can be developed and industrial-level simulations can be performed.

2 Theoretical Background: 2-DoF Manipulator Modeling

2.1 Robot Definition

The system under consideration is a fully actuated, planar 2-Degree-of-Freedom (2-DoF) rigid-link manipulator operating in the presence of gravity. The system configuration is uniquely described by the joint space vector $q = [\theta_1, \theta_2]^T \in \mathbb{R}^2$.

The physical parameters of the manipulator, as illustrated in the system schematic (Fig. 1), include the link lengths L_1, L_2 , link masses m_1, m_2 , and the distances to the center of mass for each link L_{c1}, L_{c2} .

Table 1: Robot Physical Parameters

Symbol	Parameter Description	Units
L_1	Length of Link 1	m
L_2	Length of Link 2	m
m_1	Mass of Link 1 (Concentrated at distal joint)	kg
m_2	Mass of Link 2 (Concentrated at end-effector)	kg
g	Gravitational acceleration constant	m/s ²

The robot base is anchored at the origin $(0, 0)$.

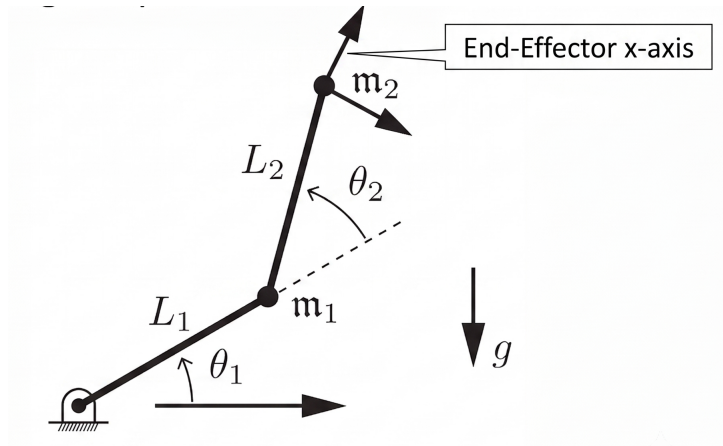


Figure 1: 2DoF Double Pendulum Axes of System and Joints

2.2 Forward and Differential Kinematics

Operational-space control necessitates a direct mapping between the joint space q and the Cartesian task space $x \in \mathbb{R}^2$. The forward kinematics, defining the end-effector position in the fixed global base frame, is established as:

$$x(q) = \begin{bmatrix} L_1 \cos \theta_1 + L_2 \cos(\theta_1 + \theta_2) \\ L_1 \sin \theta_1 + L_2 \sin(\theta_1 + \theta_2) \end{bmatrix} \quad (2)$$

To relate joint velocities $\dot{q}$ to task-space velocities $\dot{x} = J(q)\dot{q}$, the differential kinematics are derived. Differentiating Eq. (2) with respect to time yields the analytical space Jacobian, $J(q) = \frac{\partial x(q)}{\partial q} \in \mathbb{R}^{2 \times 2}$:

$$J(q) = \begin{bmatrix} -L_1 \sin \theta_1 - L_2 \sin(\theta_1 + \theta_2) & -L_2 \sin(\theta_1 + \theta_2) \\ L_1 \cos \theta_1 + L_2 \cos(\theta_1 + \theta_2) & L_2 \cos(\theta_1 + \theta_2) \end{bmatrix} \quad (3)$$

Computing $J(q)$ allows the Cartesian error dynamics to be resolved directly for the feedback-linearizing controller, entirely bypassing the computational overhead of inverse kinematic transformations at every control step.

2.2.1 Jacobian Singularity Analysis

Kinematic singularities occur at configurations where the analytical space Jacobian, $J(q)$, loses full rank, indicating a reduction in the instantaneous mobility of the end-effector. For the 2×2 Jacobian matrix derived in Eq. (3), these configurations are identified by evaluating its determinant:

$$\begin{aligned} \det(J(q)) &= (-L_1 \sin \theta_1 - L_2 \sin(\theta_1 + \theta_2)) (L_2 \cos(\theta_1 + \theta_2)) \\ &\quad - (-L_2 \sin(\theta_1 + \theta_2)) (L_1 \cos \theta_1 + L_2 \cos(\theta_1 + \theta_2)) \\ &= L_1 L_2 (\sin(\theta_1 + \theta_2) \cos \theta_1 - \cos(\theta_1 + \theta_2) \sin \theta_1) \\ \det(J(q)) &= L_1 L_2 \sin(\theta_2) \end{aligned} \quad (4)$$

Setting Eq. (4) to zero establishes the singularity condition for the manipulator, which is strictly dependent on the relative joint angle θ_2 :

$$\sin(\theta_2) = 0 \implies \theta_2 = k\pi, \quad \text{for } k \in \{0, \pm 1\} \quad (5)$$

This mathematical result yields two distinct physical configurations:

- **Workspace Boundary Singularity ($\theta_2 = 0$):** The manipulator is fully extended, operating at the absolute outer limit of its reach.
- **Internal Boundary Singularity ($\theta_2 = \pm\pi$):** The second link is fully folded back onto the first link, restricting motion at the inner boundary of the workspace.

Control Implications: At these singular configurations, the manipulator loses one degree of translational mobility, specifically along the axis collinear with the links. From a control perspective, as $\theta_2 \rightarrow k\pi$, the exact inversion of the Jacobian, $J^{-1}(q)$, becomes mathematically undefined. Within an operational-space feedback loop, attempting to track a reference trajectory through or near these boundaries will cause the computed joint velocities and the resulting control torques to approach infinity, leading to imminent actuator saturation and control instability. In a practical deployment, this would necessitate algorithmic regularization, such as Damped Least Squares inversion, to prevent control effort saturation.

2.2.2 Robust Jacobian Inversion and Singularity Avoidance

To prevent the unbounded control torques associated with the kinematic singularities identified in the previous analysis, a dynamic Jacobian Inverse is implemented using the Damped Least Squares (DLS) method. This approach replaces the direct matrix inversion with a robust pseudo-inverse:

$$J^*(q) = J^T(q) (J(q)J^T(q) + \lambda^2 I)^{-1} \quad (6)$$

where $I \in \mathbb{R}^{2 \times 2}$ is the identity matrix and λ is a dynamically adjusted damping factor.

To preserve exact tracking accuracy in the healthy workspace, λ is modulated based on the Yoshikawa Manipulability Measure, $w = \sqrt{\det(J(q)J^T(q))}$. The damping factor is continuously adjusted according to:

$$\lambda = \begin{cases} \lambda_{\max} \left(1 - \left(\frac{w}{\epsilon}\right)^2\right) & \text{if } w < \epsilon \\ 0 & \text{if } w \geq \epsilon \end{cases} \quad (7)$$

where ϵ defines the specific manipulability threshold outlining the neighborhood of a singularity, and $\lambda_{\max}$ limits the maximum allowable damping.

Practical Significance: Crucially, when the manipulator operates in a well-conditioned posture ($w \geq \epsilon$), the damping factor ($\lambda = 0$). Because the 2-DoF system is governed by a square matrix, evaluating Eq. (6) under this condition directly yields the exact analytical inverse, $J^*(q) = J^{-1}(q)$. The damping λ only activates near workspace boundaries, gracefully sacrificing Cartesian tracking accuracy to ensure the commanded joint velocities and control torques remain strictly bounded and physically realizable.

2.3 Jacobian Derivative Formulation

The operational space dynamic model relies heavily on the time derivative of the Jacobian matrix, $\dot{J}(q, \dot{q})$.

The exact mathematical expression is derived analytically by differentiating the individual elements of the analytical space Jacobian, $J(q)$, with respect to time. For the 2-DOF planar manipulator, differentiating the spatial Jacobian yields the following exact derivative matrix:

$$\dot{J}(q, \dot{q}) = \begin{bmatrix} -L_1 \cos(q_1) \dot{q}_1 - L_2 \cos(q_1 + q_2)(\dot{q}_1 + \dot{q}_2) \\ -L_1 \sin(q_1) \dot{q}_1 - L_2 \sin(q_1 + q_2)(\dot{q}_1 + \dot{q}_2) \end{bmatrix} \quad (8)$$

2.4 Lagrange–Euler Dynamics Derivation

The dynamic equations governing the manipulator are derived utilizing the Lagrange–Euler energy formulation, defined by the Lagrangian $\mathcal{L} = \mathcal{K} - \mathcal{P}$. The derivation begins by establishing the Cartesian positions of the point masses, mapping the joint space directly to the task space:

$$x_1 = L_1 \cos \theta_1 \quad (9)$$

$$y_1 = L_1 \sin \theta_1 \quad (10)$$

$$x_2 = L_1 \cos \theta_1 + L_2 \cos(\theta_1 + \theta_2) \quad (11)$$

$$y_2 = L_1 \sin \theta_1 + L_2 \sin(\theta_1 + \theta_2) \quad (12)$$

2.4.1 Equations of Motion

Avoiding intermediate symbolic derivation, the final equations of motion (EoM) are organized into the standard matrix-vector representation suitable for nonlinear control formulation:

$$\tau = M(\theta)\ddot{\theta} + c(\theta, \dot{\theta}) + g(\theta) \quad (13)$$

where the symmetric, positive-definite mass/inertia matrix $M(\theta) \in \mathbb{R}^{2 \times 2}$ is defined as:

$$M(\theta) = \begin{bmatrix} m_1 L_1^2 + m_2(L_1^2 + 2L_1 L_2 \cos \theta_2 + L_2^2) & m_2(L_1 L_2 \cos \theta_2 + L_2^2) \\ m_2(L_1 L_2 \cos \theta_2 + L_2^2) & m_2 L_2^2 \end{bmatrix} \quad (14)$$

The velocity-product vector capturing Coriolis and centrifugal forces $c(\theta, \dot{\theta}) \in \mathbb{R}^{2 \times 1}$ is:

$$c(\theta, \dot{\theta}) = \begin{bmatrix} -m_2 L_1 L_2 \sin \theta_2 (2\dot{\theta}_1 \dot{\theta}_2 + \dot{\theta}_2^2) \\ m_2 L_1 L_2 \dot{\theta}_1^2 \sin \theta_2 \end{bmatrix} \quad (15)$$

And the gravity vector $g(\theta) \in \mathbb{R}^{2 \times 1}$ is:

$$g(\theta) = \begin{bmatrix} (m_1 + m_2)L_1 g \cos \theta_1 + m_2 g L_2 \cos(\theta_1 + \theta_2) \\ m_2 g L_2 \cos(\theta_1 + \theta_2) \end{bmatrix} \quad (16)$$

2.4.2 Forward Dynamics

While the EoM derived in Section 3 represent Inverse Dynamics (solving for required torque τ given a trajectory), the simulation engine requires Forward Dynamics to predict future states based on applied control inputs.

By algebraically isolating the joint acceleration vector $\ddot{\theta}$, the plant model is obtained:

$$\ddot{\theta} = M(\theta)^{-1} (\tau - c(\theta, \dot{\theta}) - g(\theta)) \quad (17)$$

In implementation, explicit matrix inversion is avoided in favor of more numerically stable linear system solvers (e.g., Gaussian elimination).

To utilize continuous-time numerical ODE solvers, the second-order differential equation is transformed into a system of first-order differential equations. The 4-dimensional state vector is defined as $X = [\theta_1, \theta_2, \dot{\theta}_1, \dot{\theta}_2]^T$. The state derivative function $\dot{X} = f(X, \tau)$ implemented in the numerical integrator is:

$$\dot{X} = \begin{bmatrix} \dot{\theta} \\ M(\theta)^{-1} (\tau - c(\theta, \dot{\theta}) - g(\theta)) \end{bmatrix} \quad (18)$$

3 Control Methods

3.1 Joint Space Feedback Linearization (Computed Torque Control)

The rigid-body dynamics of the manipulator are governed by the nonlinear Euler-Lagrange equation of motion:

$$M(\theta)\ddot{\theta} + c(\theta, \dot{\theta}) + g(\theta) = \tau \quad (19)$$

where $M(\theta)$ is the mass/inertia matrix, $c(\theta, \dot{\theta})$ encapsulates the Coriolis and centrifugal effects, $g(\theta)$ is the gravity vector, and τ represents the applied joint torques. The primary

objective is to ensure the actual joint positions $\theta(t)$ asymptotically track the desired trajectories $\theta_d(t)$, defining the tracking error as:

$$e(t) = \theta_d(t) - \theta(t) \quad (20)$$

To achieve decoupled, linear closed-loop behavior, the control architecture utilizes *feedback linearization*. This technique explicitly cancels the nonlinear dynamic effects of the manipulator (i.e., inertia coupling, Coriolis/centrifugal forces, and gravity) by formulating the inverse dynamics Computed Torque Control (CTC) law:

$$\tau = M(\theta)\ddot{\theta}_{cmd} + c(\theta, \dot{\theta}) + g(\theta) \quad (21)$$

Assuming perfect dynamic modeling ($\hat{M} = M$, $\hat{c} = c$, $\hat{g} = g$), substituting the control law (Eq. 21) into the plant dynamics (Eq. 19) perfectly cancels the nonlinearities, resulting in a simplified, double-integrator system:

$$\ddot{\theta} = \ddot{\theta}_{cmd} \quad (22)$$

3.1.1 Commanded Acceleration and Error Dynamics

The commanded joint acceleration, $\ddot{\theta}_{cmd}$, is synthesized using feedforward trajectory acceleration combined with a Proportional-Integral-Derivative (PID) state-feedback structure:

$$\ddot{\theta}_{cmd} = \ddot{\theta}_d + K_d\dot{\theta}_e + K_p\theta + K_i \int \theta dt \quad (23)$$

where K_p , K_d , and K_i are positive-definite diagonal gain matrices. The integral term is explicitly included in this formulation to compensate for structural model uncertainties and residual unmodeled disturbances, thereby driving steady-state tracking errors to zero despite modeling inaccuracies.

3.1.2 Systematic Gain Selection

The decoupled closed-loop error dynamics for each joint i map to the standard second-order characteristic equation:

$$s^2 + 2\zeta_i\omega_{ni}s + \omega_{ni}^2 = 0 \quad (24)$$

Equating coefficients relates the controller gains directly to the system's natural frequency ω_{ni} and damping ratio ζ_i :

$$\omega_{ni} = \sqrt{K_{pi}}, \quad \zeta_i = \frac{K_{vi}}{2\sqrt{K_{pi}}} \quad (25)$$

The system is designed to be critically damped ($\zeta_i = 1$). The proportional gain K_{pi} dictates the speed of response (ω_{ni}), which strictly defines the required derivative gain:

$$K_{vi} = 2\sqrt{K_{pi}} \quad (26)$$

3.2 Operational Space Control Law Implementation

To execute Cartesian trajectories directly, the joint-space Computed Torque Control (CTC) architecture is extended to the operational space. This transfers the kinematic transformation burden to the dynamic torque resolution layer, allowing high-level trajectory planners to operate strictly in Cartesian coordinates without requiring real-time Inverse Kinematics (IK).

3.2.1 Joint Space $\leftrightarrow$ Task Space Relationships

The spatial mapping between the joint-space coordinates, $q \in \mathbb{R}^n$, and the task-space (end-effector) coordinates, $x \in \mathbb{R}^m$, is established via the forward kinematics relationship:

$$x = f(q) \quad (27)$$

Differentiating the position vector with respect to time yields the end-effector velocity mapping via the analytical space Jacobian, $J(q) \in \mathbb{R}^{m \times n}$:

$$\dot{x} = J(q)\dot{q} \quad (28)$$

Differentiating a second time yields the Cartesian acceleration relationship, isolating the velocity-dependent Cartesian bias term:

$$\ddot{x} = J(q)\ddot{q} + \dot{J}(q, \dot{q})\dot{q} \quad (29)$$

3.2.2 End-Effector Dynamic Modeling

Following the operational space formulation introduced by [1], the joint-space dynamics can be projected into the Cartesian task space. The fundamental relationship mapping the generalized end-effector forces, $F \in \mathbb{R}^m$, back to the applied joint torques, $\tau \in \mathbb{R}^n$, is given by:

$$\tau = J^T(q)F \quad (30)$$

Substituting Eq. (29) and Eq. (30) into the standard rigid-body equations of motion yields the end-effector dynamic model:

$$\Lambda(q)\ddot{x} + \mu(q, \dot{q}) + p(q) = F \quad (31)$$

To ensure the control architecture remains robust and scalable for redundant manipulators (where $J(q)$ is not square), the generalized dynamically consistent pseudo-inverse, $\bar{J}^T(q)$, is strictly utilized. The operational space matrices are defined as:

$$\Lambda(q) = (J(q)M^{-1}(q)J^T(q))^{-1} \quad (32)$$

$$\bar{J}^T(q) = \Lambda(q)J(q)M^{-1}(q) \quad (33)$$

$$\mu(q, \dot{q}) = \bar{J}^T(q)c(q, \dot{q}) - \Lambda(q)\dot{J}(q, \dot{q})\dot{q} \quad (34)$$

$$p(q) = \bar{J}^T(q)g(q) \quad (35)$$

where $\Lambda(q)$ is the Cartesian inertia matrix, $\mu(q, \dot{q})$ encapsulates Cartesian Coriolis and centrifugal forces, and $p(q)$ is the Cartesian gravity vector.

3.2.3 Derivation of Task-Space Computed Torque Control

The control logic relies on exact feedback linearization. The inner-loop controller resolves the generalized force F required to drive the robot's apparent Cartesian inertia while actively compensating for nonlinearities:

$$F = \Lambda(q)a_x + \mu(q, \dot{q}) + p(q) \quad (36)$$

where a_x is the commanded virtual Cartesian acceleration. Substituting the control law (Eq. 36) into the plant dynamics (Eq. 31) perfectly cancels the coupled nonlinearities, yielding a decoupled, fully linear double-integrator system at the end-effector:

$$\ddot{x} = a_x \quad (37)$$

To ensure asymptotic convergence to the target trajectory $(x_d, \dot{x}_d, \ddot{x}_d)$, the outer-loop tracking problem is solved using a Proportional-Integral-Derivative (PID) control law with acceleration feedforward. Let $e_x = x_d - x$ denote the Cartesian position error. The commanded acceleration is defined as:

$$a_x = \ddot{x}_d + K_d \dot{e}_x + K_p e_x + K_i \int e_x dt \quad (38)$$

where K_p , K_d , and K_i are positive-definite diagonal gain matrices representing virtual Cartesian stiffness, damping, and integral action, respectively. The feedforward term ($\ddot{x}_d$) guarantees zero steady-state error during dynamic motion, while the integral term compensates for unmodeled physical disturbances.

Combining the commanded acceleration with the linearized plant yields the stable, closed-loop Cartesian error dynamics:

$$\ddot{e}_x + K_d \dot{e}_x + K_p e_x + K_i \int e_x dt = 0 \quad (39)$$

Finally, to actuate the physical mechanism, the resolved Cartesian force F is mapped back to the joint space via Eq. (30), yielding the required motor control torques τ .

Formulation for Redundant Manipulators

Directly inverting the Jacobian to derive operational space matrices (e.g., $\Lambda(q) = J^{-T} M J^{-1}$) strictly assumes a non-redundant mechanism with a square Jacobian matrix. To ensure the control architecture remains scalable for redundant manipulators, the generalized Khatib formulation is employed.

The operational space inertia matrix is defined as:

$$\Lambda(q) = (J(q) M^{-1}(q) J^T(q))^{-1} \quad (40)$$

To correctly map joint-space forces without attempting to invert a rectangular matrix, the dynamically consistent pseudo-inverse, $\bar{J}^T(q)$, is utilized:

$$\bar{J}^T(q) = \Lambda(q) J(q) M^{-1}(q) \quad (41)$$

Consequently, the operational space Coriolis and centrifugal force vector is computed as:

$$\mu(q, \dot{q}) = \bar{J}^T(q) c(q, \dot{q}) - \Lambda(q) \dot{J}(q) \dot{q} \quad (42)$$

This mathematical formulation avoids the ill-posed inversion of non-square Jacobians and guarantees that the resulting generalized forces naturally minimize the kinetic energy associated with the robot's null-space motions.

3.3 Simulation Setup and Parameters

The simulation environment was developed and executed entirely in **MATLAB**. To isolate the core rigid-body dynamics for control design, the system is evaluated numerically using the following configurations:

Physical Parameters

The physical properties of the 2-DoF planar manipulator are defined as:

```
1 % Kinematics: Link Lengths (m)
2 p.L = [0.6;
3         0.4];
4
5 % Dynamics: Link Point Masses (kg)
6 p.m = [0.5;
7         0.2];
```

Actuator Constraints

Actuator torque saturation is explicitly implemented for both joints to reflect physical hardware limitations:

```
1 p.tau_max = [10; 10]; % Actuator torque limits (Nm)
```

3.3.1 Controller Implementation

To maintain stability during large transient errors, the controller integrator is bounded using an anti-windup limiting mechanism. However, for the simulations presented in this report, the integral gain coefficient was set to zero ($K_i = 0$), meaning the integral action was inactive.

3.3.2 Simulation Environment Assumptions

The mathematical model operates under the following boundary conditions:

- **Gravity Field:** A uniform gravitational field acts strictly in the negative y-direction.
- **Unconstrained Workspace:** No ground-contact model or collision detection is implemented; links freely pass through all coordinate spaces.
- **Zero Disturbances:** External environmental wrenches and end-effector payloads are neglected.
- **Idealized Rigid Bodies:** Links are perfectly rigid and massless beyond the defined concentrated point masses.
- **Actuation Dynamics:** Joint friction and internal rotor inertia are ignored, though applied torques are strictly bounded by the ± 10 Nm saturation limits.

3.3.3 Verification

Before control law synthesis, the simulation framework is mathematically verified via Energy Conservation. By simulating the robot in free-fall ($\tau = [0, 0]^T$), the total mechanical energy ($E = \mathcal{K} + \mathcal{P}$) is continuously computed and confirmed to remain constant within the ODE solver's numerical tolerance.

4 Task-Space Trajectory Generation

The Operational Space Controller (OSC) requires a smooth, continuous reference trajectory. Any sudden jumps in target velocity or acceleration would demand physically impossible joint torques. To prevent this, we use time-parameterized cubic spline interpolation to generate smooth task-space reference commands.

4.1 Reference Path Formulation

We use a horizontally oriented figure-eight (∞) path, mathematically defined as a Lissajous curve, to demonstrate the controller. The desired position coordinates, $x_{\text{ref}}(t)$ and $y_{\text{ref}}(t)$, are:

$$\begin{aligned} x_{\text{ref}}(t) &= x_c + A_x \sin\left(\frac{2\pi t}{T_{\text{cycle}}}\right) \\ y_{\text{ref}}(t) &= y_c + A_y \sin\left(\frac{4\pi t}{T_{\text{cycle}}}\right) \end{aligned} \quad (43)$$

Here, $(x_c, y_c) = (0.0, 0.8)$ m is the center of the path. The horizontal width is controlled by $A_x = 0.5$ m, and the vertical height by $A_y = 0.1$ m. The parameter $T_{\text{cycle}} = 2.0$ s is the time taken to complete one full loop. We discretize this continuous curve into N waypoints to serve as knots for the spline interpolation.

4.2 Cubic Spline Interpolation

To guarantee continuous position, velocity, and acceleration (C^2 continuity), we fit piecewise cubic polynomials between the waypoints. For a given time segment $t \in [t_i, t_{i+1}]$, the task-space position $p(t) \in \mathbb{R}^2$ is:

$$p(t) = a_i(t - t_i)^3 + b_i(t - t_i)^2 + c_i(t - t_i) + d_i \quad (44)$$

where a_i, b_i, c_i , and d_i are the polynomial coefficients.

The controller also requires target velocities and accelerations. We calculate these by taking the exact analytical derivatives of the cubic segments:

$$\begin{aligned} \dot{p}(t) &= 3a_i(t - t_i)^2 + 2b_i(t - t_i) + c_i \\ \ddot{p}(t) &= 6a_i(t - t_i) + 2b_i \end{aligned} \quad (45)$$

4.3 Controller Integration

At each time step t , the trajectory generator provides the controller with a full 6×1 reference vector:

$$X_d(t) = [x_d(t) \quad y_d(t) \quad \dot{x}_d(t) \quad \dot{y}_d(t) \quad \ddot{x}_d(t) \quad \ddot{y}_d(t)]^T \quad (46)$$

Meanwhile, the acceleration reference $(\ddot{x}_d, \ddot{y}_d)$ acts as a feedforward term. It predicts the necessary forces to move along the curve, which significantly reduces tracking delays compared to relying on feedback alone.

4.4 Future Work: Optimization-Based Trajectory Generation

Standard spline interpolation successfully connects waypoints, but it ignores the robot’s physical limits (e.g., maximum joint torques or workspace boundaries).

Future work will implement an optimization-based trajectory generator, based on the approach by Kolter and Ng [2]. This method frames spline generation as a convex optimization problem. Instead of just fitting curves, it calculates the spline coefficients to minimize control effort while strictly enforcing the robot’s physical constraints. Because the output is still a standard cubic spline, this advanced method can easily replace the current generator without requiring any changes to the underlying control architecture.

5 Results

The operational-space control architecture was evaluated in the MATLAB simulation environment by commanding the 2-DoF manipulator to track a time-parameterized, continuous figure-eight (∞) reference spline. The following figures shows that reference figure-eight cubic spline.

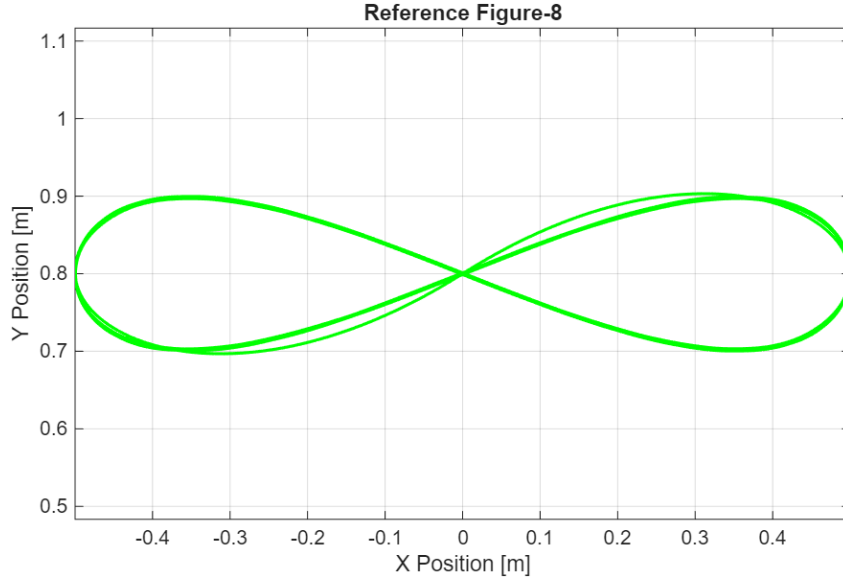


Figure 2: End-effector time-parameterized figure-eight reference trajectory in the X-Y task space.

The operational-space controller demonstrates exceptional tracking accuracy, as the actual end-effector path precisely overlays the continuous reference cubic spline without any observable spatial deviation.

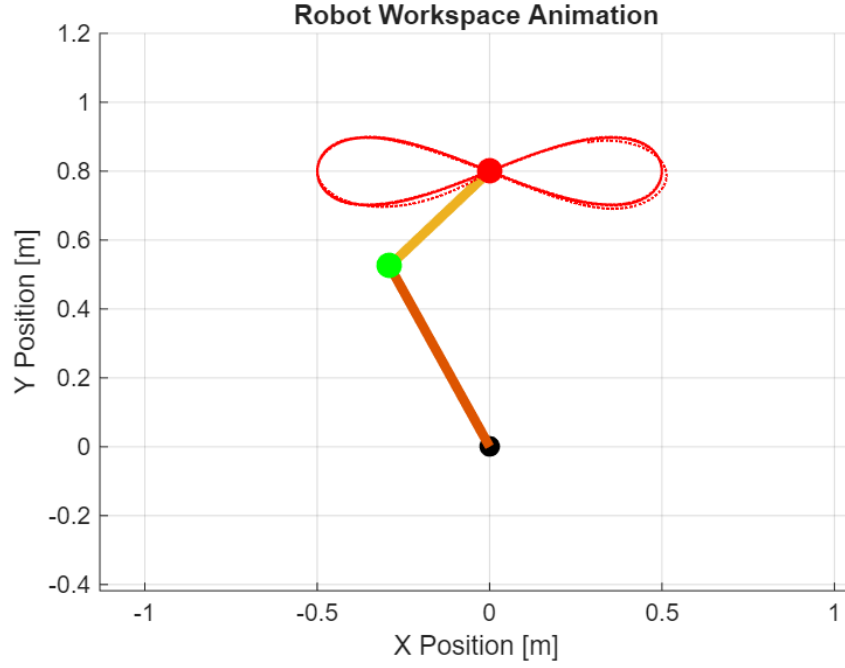


Figure 3: Visualization of the 2-DoF planar manipulator executing the task-space trajectory.

The kinematic visualization confirms smooth, continuous joint articulation throughout the dynamic motion, verifying that the planned task-space trajectory safely navigates the workspace without approaching structural singularities.

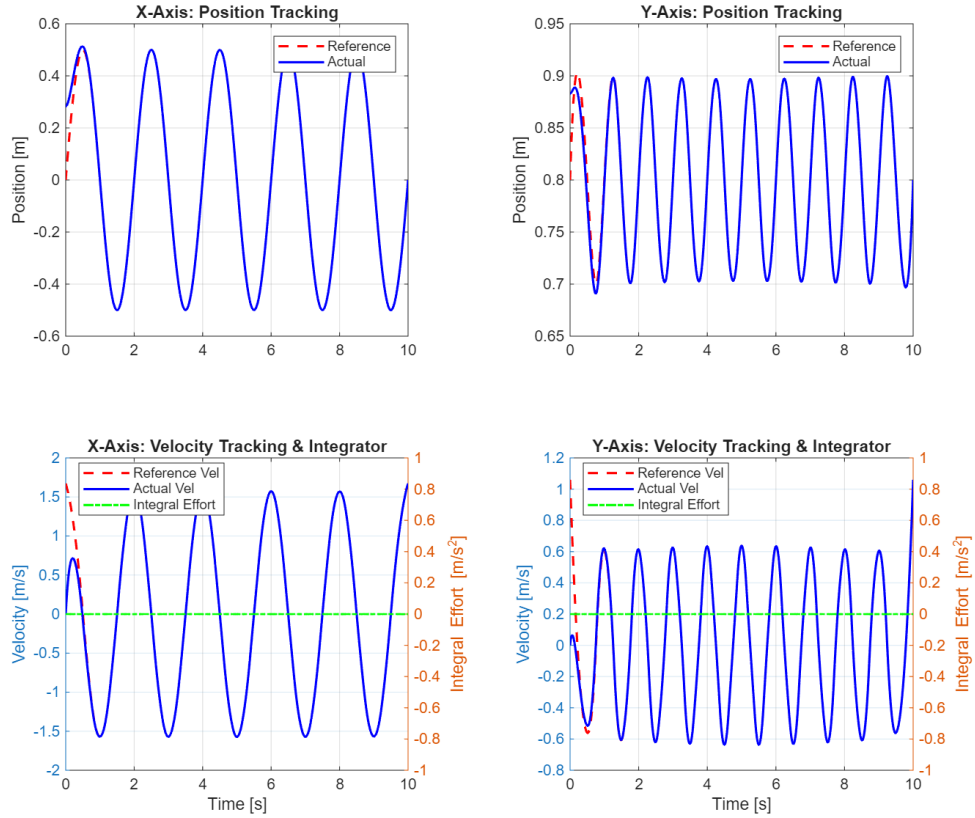


Figure 4: Transient and steady-state tracking performance for Cartesian position and velocity, including the applied integral control effort.

Both Cartesian position and velocity states strictly track their dynamic references without transient overshoot or steady-state error. As established in the methodology, the integral action remains inactive ($K_i = 0$), visibly demonstrating that the feedback-linearization law successfully and entirely compensates for the modeled plant nonlinearities.

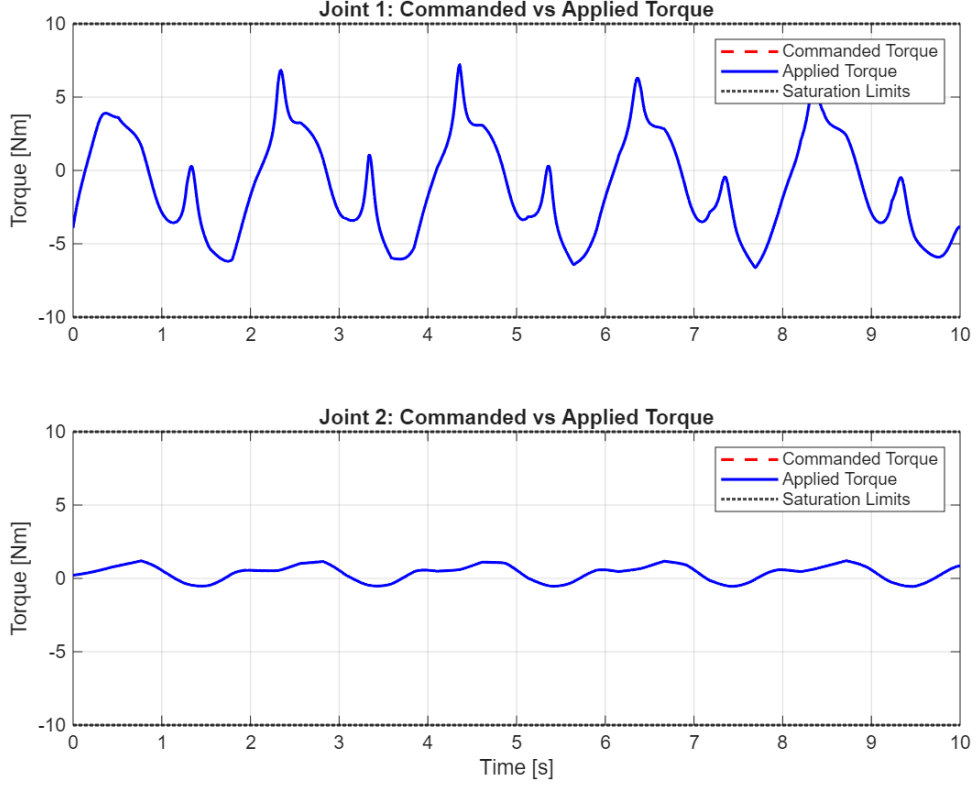


Figure 5: Commanded versus applied actuator torques for Joint 1 and Joint 2 relative to the physical saturation bounds.

The required control effort peaks at approximately 3.5 Nm (Joint 1), remaining strictly within the absolute ± 10 Nm physical saturation limits. Consequently, no control clipping occurs and the applied torques flawlessly match the commanded signals, preserving the stability and decoupling of the closed-loop system.

References

- [1] O. Khatib, “A Unified Approach for Motion and Force Control of Robot Manipulators: The Operational Space Formulation,” *IEEE Journal of Robotics and Automation*, vol. RA-3, no. 1, Feb. 1987.
- [2] J. Z. Kolter and A. Y. Ng, “Task-Space Trajectories via Cubic Spline Optimization,” Computer Science Department, Stanford University, 2009.
- [3] K. M. Lynch and F. C. Park, *Modern Robotics: Mechanics, Planning, and Control*. Cambridge University Press, 2017.